

[START HERE](#)

Index, Journey Map & Skill Assessment

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



System Design Master Roadmap — 9 Years of Experience Edition

A complete High-Level System Design learning and interview-preparation program, built for a backend engineer/tech lead with 9 years of production experience (PHP/Laravel, Node.js, MySQL, PostgreSQL, MongoDB, Redis, Kafka, Elasticsearch, GraphQL, Docker, AWS, Kubernetes/EKS/ECS, Terraform, Prometheus, Grafana, OpenTelemetry) who is strong in engineering execution but new to formal System Design interviewing — targeting Tier-2 to Tier-1 product companies across India and Dubai/UAE.

Compiled August 2026. Research current as of this date — verify company-specific interview process details against fresh sources closer to your actual interview, since these change.

My System Design Journey

```
Fundamentals
  ↓
Networking
  ↓
Scalability
  ↓
Databases
  ↓
Caching
  ↓
Messaging
  ↓
Distributed Systems
  ↓
Microservices
  ↓
Cloud
  ↓
Reliability
  ↓
Security
  ↓
Observability
  ↓
Architecture Patterns
  ↓
System Design Problems
  ↓
Mock Interviews
  ↓
Company-Specific Preparation
  ↓
Amazon / Careem / Tier-1 Ready
```

1. Current Skill Assessment

Based on your background, here's an honest read of where you're starting from — this roadmap is calibrated to it, not to a generic beginner.

What you almost certainly already know, even if you haven't named it formally:

- Real operational instincts for caching (Redis), messaging (Kafka), and search (Elasticsearch) — you've felt consumer lag, cache invalidation pain, and slow queries firsthand, which most candidates only know from a textbook.
- Container orchestration and infrastructure-as-code (Kubernetes/EKS/ECS, Terraform) — service discovery, health checks, and deployment concerns (Chapter 8) will feel like description, not new information.
- Observability tooling (Prometheus, Grafana, OpenTelemetry) — Chapter 11 is largely a

vocabulary/framing exercise for you, not new content; this is a genuine, underused differentiator most candidates don't have.

- Polyglot persistence in practice (MySQL, PostgreSQL, MongoDB) — you've likely already made SQL-vs-NoSQL calls under real constraints, even if not through the explicit decision framework in Chapter 4.6.

What's genuinely new and needs deliberate study, not just "I'll pick it up":

- **Structured estimation under time pressure** (Chapter 3) — doing back-of-envelope math cleanly, out loud, in 5 minutes is a specific, practiced skill distinct from the capacity planning you may have done at work with more time and real data available.

- **The interview-specific communication layer** (Chapter 14) — narrating your reasoning, inviting challenge, and explicitly stating trade-offs unprompted is a *performance* skill on top of technical knowledge, and it's the single highest-leverage gap for experienced engineers new to this format.

- **Distributed-systems theory vocabulary** (Chapter 6) — quorum math, consensus, CAP/PACELC precisely stated, Saga vs. 2PC — you've likely worked *around* these problems in practice without needing the formal names; interviews expect the formal names and precise reasoning.

- **Fintech-specific patterns** (Chapter 20) — double-entry ledgers, reconciliation, chargeback handling — genuinely new domain knowledge unless you've worked in payments specifically, and high-priority given your stated target companies.

- **The unfamiliar-product-from-first-principles muscle** (Chapter 29.2) — this is what separates "I know these 30 memorized answers" from true interview readiness, and it only comes from mock-interview repetition (Chapter 28), not reading.

Bottom line: you're not starting from zero — you're starting from strong practical fluency with a real gap in formal vocabulary, structured estimation, and interview performance. Expect the early chapters to move fast for you and the later ones (interview framework, problems, mocks) to be where the real work happens.

2. Complete Table of Contents

Foundations

- [Chapter 1 — Prerequisites & Networking](#) (Level 0)
- [Chapter 2 — System Design Fundamentals](#) (Level 1)
- [Chapter 3 — Capacity Estimation](#) (Level 2, 10 worked examples)
- [Chapter 4 — Core Building Blocks](#) (Level 3: LB, CDN, cache, SQL/NoSQL)
- [Chapter 5 — Database Deep Dive](#) (Level 4)
- [Chapter 6 — Distributed Systems](#) (Level 5 — the deepest chapter)
- [Chapter 7 — Messaging & Event-Driven Architecture](#) (Level 6)
- [Chapter 8 — Microservices](#) (Level 7)

- [Chapter 9 — API Design](#) (Level 8)
- [Chapter 10 — Security](#) (Level 9)
- [Chapter 11 — Observability & Operations](#) (Level 10)
- [Chapter 12 — Cloud System Design \(AWS\)](#) (Level 11)
- [Chapter 13 — Architecture Patterns](#) (Level 12, full pattern catalog)

Interview Craft

- [Chapter 14 — Interview Framework & Communication](#) (Levels 13, 21, 22, 23 — the universal framework, communication scripts, seniority calibration, common mistakes)
- [Chapter 15 — Diagramming Guide](#) (Level 14)

Practice Problems

- [Chapter 16 — Beginner Problems](#) (6 problems)
- [Chapter 17 — Intermediate Problems](#) (10 problems)
- [Chapter 18 — Advanced Problems](#) (14 problems — all 30 problems complete across Ch.16-18, Level 15)

Company & Domain Specialization

- [Chapter 19 — Company-Specific Preparation](#) (Level 16: Amazon, Google, Microsoft, Meta, Uber, Atlassian, Airbnb, Flipkart, Walmart, Razorpay, PhonePe, CRED, Meesho, Swiggy, Zomato, Zepto, Careem, Noon, Talabat, Deliveroo, Tabby, Tamara, PayBy, Ziina, Magnati)
- [Chapter 20 — FinTech System Design](#) (Level 17)
- [Chapter 21 — Real-Time System Design](#) (Level 18)
- [Chapter 22 — Search Systems](#) (Level 19)
- [Chapter 23 — Performance Engineering](#) (Level 20)

Resources & Practice

- [Chapter 24 — Resource Library](#) (Level 24: EN/Hindi YouTube, blogs, books, courses)
- [Chapter 25 — Study Plans](#) (Levels 25-27: 16-week plan, daily routines, spaced revision)
- [Chapter 26 — Cheat Sheet & Decision Matrices](#) (Level 28 + decision matrices)
- [Chapter 27 — Question Bank](#) (Level 29: 270+ questions)
- [Chapter 28 — Mock Interview Program](#) (Level 30: 20 mocks)
- [Chapter 29 — Evaluation Rubric & Final Framework](#) (Levels 31-32: 100-point rubric + the one-page "design any product" decision tree)

3. How to Use This Roadmap

1. **Weeks 1-8 (Foundations):** read Chapters 1-15 in order, following the [16-week study plan](#). Given your background, you can likely move faster than 15 hours/week through Chapters 1, 7, 8, and 11 specifically — don't force artificial time on material you already operate daily; redirect that saved time to Chapter 6 and Chapter 14.
 2. **Weeks 9-13 (Problems & Specialization):** work through Chapters 16-23, doing the problems *solo and timed* before reading each chapter's worked answer — this is non-negotiable for the framework to actually stick.
 3. **Weeks 14-16 (Practice & Company Prep):** run the [20 mock interviews](#), score yourself against the [rubric](#), and layer in [company-specific prep](#) for your actual target companies.
 4. **The week before any real interview:** re-read the [Cheat Sheet](#) daily, run 1-2 light mocks, and stop consuming new material — taper, don't cram.
 5. **Ongoing:** keep the [spaced revision schedule](#) running in the background for anything you haven't touched in 2+ weeks.
-

4. Final Success Criteria

By the end of this roadmap, you should be able to confidently design, from first principles and without notes: a URL shortener, WhatsApp, Instagram, YouTube, Netflix, Uber, Careem, Amazon, a food delivery platform, a payment gateway, a digital wallet, a notification system, a search system, a rate limiter, a distributed cache, a ride-matching engine, a ticket-booking system, an e-commerce platform, and real-time location tracking — and, more importantly, **any unfamiliar system design question**, using the framework in [Chapter 29.2](#).

Good luck — you have a stronger starting foundation than you probably give yourself credit for. The gap between where you are and Tier-1-ready is real but entirely closeable with the structured practice this roadmap lays out.